

LLD Case Study 4: Car Rental System (Hertz / Zoomcar)

Target Patterns: Factory Pattern, Strategy Pattern, State Pattern

Key Engineering Focus: Multi-branch vehicle inventory, dynamic pricing algorithms, vehicle lifecycle states, and reservation management.

1. Problem Statement & Functional Requirements

Design an automated Car Rental Management platform operating across multiple cities and airport branches.

Requirements:

- Vehicle Hierarchy (Factory Pattern):** Manage diverse vehicle classes (Economy Hatchback, Compact SUV, Luxury Sedan, Electric Vehicle) with distinct specifications and rates.
 - Dynamic Pricing (Strategy Pattern):** Calculate rental costs using swappable pricing algorithms (Base Daily Rate, Surge/Peak Season Pricing, Distance/Mileage Model).
 - Reservation State Machine (State Pattern):** Reservations progress from Pending $\rightarrow$ Confirmed $\rightarrow$ PickedUp $\rightarrow$ Completed (or Cancelled).
 - Multi-Location Inventory:** Allow vehicle pickup at one branch and return at a different branch with inter-branch inventory rebalancing.
-

2. Architecture & UML Class Diagram

```
classDiagram
    class VehicleType {
        <<enumeration>>
        ECONOMY
        SUV
        LUXURY
        ELECTRIC
    }

    class Vehicle {
        +String licensePlate
        +VehicleType type
        +float baseDailyRate
        +bool isAvailable
    }

    class PricingStrategy {
        <<interface>>
        +calculatePrice(Vehicle vehicle, int days, float milesDriven) float
    }

    class DailyPricingStrategy {
        +calculatePrice(Vehicle vehicle, int days, float milesDriven) float
    }

    class SurgePricingStrategy {
        +calculatePrice(Vehicle vehicle, int days, float milesDriven) float
    }

    class ReservationStatus {
        <<enumeration>>
        CONFIRMED
        ACTIVE
        COMPLETED
        CANCELLED
    }

    class Reservation {
        +String reservationId
    }
```

```
+Vehicle vehicle
+Customer customer
+ReservationStatus status
+PricingStrategy pricing
+calculateTotal() float
+pickupVehicle() void
+returnVehicle() void
}

VehicleFactory ..> Vehicle : Creates
Reservation o-- Vehicle : Associates
Reservation o-- PricingStrategy : Uses Strategy
```

3. Production-Grade Python Implementation

```
import uuid
from enum import Enum, auto
from abc import ABC, abstractmethod
from typing import Dict, List, Optional

# --- Enums & Vehicle Entities ---
class VehicleType(Enum):
    ECONOMY = auto()
    SUV = auto()
    LUXURY = auto()
    ELECTRIC = auto()

class Vehicle:
    def __init__(self, license_plate: str, vehicle_type: VehicleType, base_daily_rate: float):
        self.license_plate = license_plate
        self.vehicle_type = vehicle_type
        self.base_daily_rate = base_daily_rate
        self.is_available = True

# --- Factory Pattern: Vehicle Creation ---
class VehicleFactory:
    @staticmethod
    def create_vehicle(v_type: VehicleType, plate: str) -> Vehicle:
        rates = {
            VehicleType.ECONOMY: 45.0,
            VehicleType.SUV: 85.0,
            VehicleType.LUXURY: 180.0,
            VehicleType.ELECTRIC: 95.0,
        }
        return Vehicle(license_plate=plate, vehicle_type=v_type, base_daily_rate=rates[v_type])

# --- Strategy Pattern: Pricing Engines ---
class PricingStrategy(ABC):
    @abstractmethod
    def calculate_price(self, vehicle: Vehicle, rental_days: int, miles_driven: float) -> float:
        pass

class DailyPricingStrategy(PricingStrategy):
    """Standard flat daily rate with unlimited mileage."""
```

```

    def calculate_price(self, vehicle: Vehicle, rental_days: int, miles_driven: float) →
        return vehicle.base_daily_rate * rental_days

class SurgePricingStrategy(PricingStrategy):
    """Applies a surge multiplier during high demand holidays."""
    def __init__(self, multiplier: float = 1.4):
        self.multiplier = multiplier

    def calculate_price(self, vehicle: Vehicle, rental_days: int, miles_driven: float) →
        return (vehicle.base_daily_rate * rental_days) * self.multiplier

class MileageHybridStrategy(PricingStrategy):
    """Lower daily rate + per-mile charge."""
    def calculate_price(self, vehicle: Vehicle, rental_days: int, miles_driven: float) →
        base = (vehicle.base_daily_rate * 0.7) * rental_days
        mileage_fee = miles_driven * 0.25
        return base + mileage_fee

# --- Reservation Lifecycle ---
class ReservationStatus(Enum):
    CONFIRMED = auto()
    ACTIVE = auto()
    COMPLETED = auto()
    CANCELLED = auto()

class Reservation:
    def __init__(self, customer_name: str, vehicle: Vehicle, days: int, pricing: PricingS
        self.reservation_id = str(uuid.uuid4())[:8]
        self.customer_name = customer_name
        self.vehicle = vehicle
        self.rental_days = days
        self.pricing_strategy = pricing
        self.status = ReservationStatus.CONFIRMED
        self.vehicle.is_available = False # Reserve vehicle

    def pickup(self) → None:
        if self.status ≠ ReservationStatus.CONFIRMED:
            raise ValueError("Only confirmed reservations can be picked up.")
        self.status = ReservationStatus.ACTIVE
        print(f"[Rental] Customer '{self.customer_name}' picked up vehicle {self.vehicle.

    def complete_return(self, miles_driven: float) → float:

```

```
        if self.status != ReservationStatus.ACTIVE:
            raise ValueError("Cannot return vehicle that is not currently active.")
        self.status = ReservationStatus.COMPLETED
        self.vehicle.is_available = True

        total_bill = self.pricing_strategy.calculate_price(self.vehicle, self.rental_days)
        print(f"[Rental] Vehicle {self.vehicle.license_plate} returned. Total Bill: ${total_bill}")
        return total_bill

# --- Verification Driver ---
if __name__ == "__main__":
    # Create vehicles using Factory
    suv = VehicleFactory.create_vehicle(VehicleType.SUV, "CA-992-XYZ")
    luxury = VehicleFactory.create_vehicle(VehicleType.LUXURY, "NY-771-LUX")

    # 1. Standard Daily Rental
    res1 = Reservation("David", suv, days=3, pricing=DailyPricingStrategy())
    res1.pickup()
    bill1 = res1.complete_return(miles_driven=120)

    # 2. Holiday Surge Rental
    res2 = Reservation("Elena", luxury, days=2, pricing=SurgePricingStrategy(multiplier=1.5))
    res2.pickup()
    bill2 = res2.complete_return(miles_driven=40)
```